Computers & Security 168 (2026) 104969

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

FastPoS: An efficient Proof of Storage scheme with polynomial commitments for fog–cloud IoT systems

Yuting An [a], Helei Cui [a,*], Hao Zeng [a], Xiaoning Liu [b], Bin Guo [a], Zhiwen Yu [a,c]

[a] School of Computer Science, Northwestern Polytechnical University, Xi'an, China
[b] School of Computing Technologies, RMIT University, Melbourne, Australia
[c] College of Computer Science and Technology, Harbin Engineering University, Harbin, China

ARTICLE INFO

Keywords:
Internet of Things
Proof of Storage
Fog–cloud computing
Data recoverability
Polynomial commitments

ABSTRACT

With the increasing prevalence of data outsourcing, Proof of Storage (PoS) has become a fundamental technique for verifying the integrity of remotely stored data. In practical Internet of Things (IoT) deployments, however, data are typically processed and encoded by fog nodes before being uploaded to the cloud. Most existing PoS schemes are designed for end-to-cloud architectures and fail to adequately address the security and accountability challenges introduced by intermediate entities. Moreover, many prior approaches focus solely on data possession without formally guaranteeing data recoverability, and often incur high computational overhead and linear verification complexity, limiting their scalability in large-scale IoT environments. To address these limitations, we propose FastPoS, a high-performance PoS scheme tailored for fog–cloud IoT systems. FastPoS integrates Reed–Solomon coding with KZG polynomial commitments to enable independent and verifiable auditing of both fog nodes and cloud servers. Meanwhile, it ensures data recoverability by cryptographically binding fog nodes to their data processing operations. By leveraging aggregated proofs, FastPoS reduces the time cost of proof verification and tag generation. As a result, it achieves sub-second verification with a 99% detection probability and provides 15–34× faster tag generation compared with existing schemes. Extensive experimental evaluations demonstrate that FastPoS significantly outperforms state-of-the-art approaches in computational efficiency, communication overhead, and scalability, making it a practical and efficient solution for large-scale, latency-sensitive IoT applications.

1. Introduction

With the advent of the big data era, utilizing cloud storage for massive datasets has become a prevalent practice. However, outsourcing data storage inevitably leads to a decoupling of data management rights from ownership, raising concerns regarding the integrity and availability of outsourced data (Kaaniche and Laurent, 2017). To address these concerns, the concept of Proof of Storage (PoS) has been introduced and extensively studied in recent years (Ateniese et al., 2007; Juels and Kaliski, 2007; Damgård et al., 2019; Moran and Orlov, 2019; Xu et al., 2024; Lakshmi and T., 2025). PoS enables data owners to verify that their data remains securely stored and properly maintained without requiring direct access to the data itself.

Nevertheless, when applied to Internet of Things (IoT) scenarios, new challenges arise. The majority of current PoS are constructed under an end-to-cloud system model. They implicitly assume that data is outsourced directly from end devices to the cloud, treating the cloud as the sole storage entity requiring auditing and constraints. However, in typical IoT deployments, data generated by end devices is typically aggregated, encoded, or committed by intermediate fog nodes before being uploaded to the cloud for long-term storage (Tian et al., 2019; Zhang et al., 2023). This establishes an end–fog–cloud architecture in which fog nodes actively participate in the outsourcing pipeline. While traditional PoS schemes effectively constrain cloud-side deletion, they do not provide accountability for fog layer behavior. The preprocessing and commitment generation performed by fog nodes are implicitly trusted, leaving a blind spot in the security chain.

Therefore, fog node accountability is a distinct and necessary security requirement. Cloud-side deletion resistance focuses on whether stored data is retained, while fog accountability concerns whether intermediate nodes correctly execute encoding, generate valid commitments, and refrain from deleting, replacing or fabricating data before it reaches the cloud. If a fog node maliciously alters or selectively discards data during preprocessing, the cloud may faithfully store the incorrect

* Corresponding author.
E-mail addresses: anyuting@mail.nwpu.edu.cn (Y. An), chl@nwpu.edu.cn (H. Cui), hao_zeng@mail.nwpu.edu.cn (H. Zeng), xiaoning.liu@rmit.edu.au (X. Liu), guob@nwpu.edu.cn (B. Guo), zhiwenyu@nwpu.edu.cn (Z. Yu).

https://doi.org/10.1016/j.cose.2026.104969
Received 14 March 2026; Received in revised form 29 April 2026; Accepted 13 May 2026
Available online 19 May 2026

Table 1
Comparison of proof of storage schemes.

Scheme	Resists deletion attacks	Fog node accountability	Recoverability	Data verification
Tian et al. (2019)	✗	✓	✗	Zero-knowledge proofs
Anthoine et al. (2021)	✓	✗	✓	Matrix commitments
Le et al. (2023)	✓	✗	✓	Polynomial commitments
Zhang et al. (2023)	✓	✓	✗	Zero-knowledge proofs
FastPoS (Ours)	✓	✓	✓	Polynomial commitments

result and render subsequent audits meaningless. The conventional PoS constructions lack mechanisms to cryptographically bind fog-layer operations to verifiable evidence, which makes fault attribution across layers infeasible.

Some recent works attempt to expand PoS in fog–cloud architectures (Tian et al., 2019; Zhang et al., 2023). However, these schemes only confirm possession of data while not providing formal guarantees for data recoverability in the long run. This limitation is particularly critical in IoT applications, where the data is often non-renewable, generated continuously, and is very valuable for historical analysis, decision-making, and regulatory compliance. Without recoverability guarantees, corruption, deletion, or loss of data at either the fog or cloud layers could lead to irreversible information loss, thereby undermining the reliability of subsequent analytics and audits.

Finally, the efficiency of PoS schemes significantly impacts system scalability. Current solutions utilize constructions like zero-knowledge proofs or matrix commitments, which can introduce high computational costs. In large-scale deployments of IoT-based use cases with high-frequency auditing and large amounts of data, heavy-weight verification primitives have limited practical applicability. As a result, a desired scheme is required that can provide cloud-side deletion resistance, fog-node accountability, recoverability, and efficient verification at the same time.

To overcome the limitations of existing works (see Table 1), we propose FastPoS, an efficient and recoverable PoS scheme for fog–cloud IoT architectures. In contrast to conventional two-party constructions, FastPoS is explicitly designed under an end–fog–cloud architecture and includes fog nodes within the formal security boundary. The scheme is strongly deletion-resistant against malicious cloud providers and offers cryptographic accountability for fog-layer preprocessing operations. Additionally, redundancy coding can be accommodated in the commitment structure, which allows for data recovery under the specified coding assumptions.

Our main contributions are summarized as follows:

- We design a new Proof of Storage (PoS) scheme for fog–cloud IoT systems, enabling verifiable data outsourcing and auditing in hierarchical IoT architectures. The proposed scheme provides cloud-side data deletion resistance and fog-node accountability, and supports data recoverability under explicit redundancy assumptions.
- We introduce a polynomial commitment-based cryptographic mechanism that binds fog-node preprocessing operations to verifiable proofs, ensuring strong security guarantees while remaining lightweight and efficient for resource-constrained IoT environments.
- We perform formal theoretical analysis and comprehensive experimental evaluations. The results demonstrate that FastPoS achieves improvements in both computational efficiency and communication overhead. Compared with existing schemes, it reduces communication cost, improves tag generation speed by 15–34×, and enables sub-second proof verification.

2. Related works

2.1. Proof of storage in traditional cloud storage

PoS is a fundamental cryptographic technique for ensuring the integrity and availability of outsourced data in cloud storage systems. Current PoS schemes can be grouped into four representative types: Proof of Data Possession (PDP), Proof of Retrievability (PoR), Proof of Replication (PoRep), and Proof of Space–Time (PoST).

Among them, PDP and PoR are the most widely adopted foundational approaches in cloud storage auditing. Their core idea is to repeatedly execute challenge–response protocols over randomly sampled data blocks, thereby reducing the probability that a cloud server can pass audits despite data loss or deletion to a negligible level. The PDP scheme was first introduced by Ateniese et al. (2007), who proposed an RSA-based homomorphic tag construction supporting public verification. Subsequently, Juels and Kaliski (2007) presented the first PoR scheme, which combines spot-checking with error-correcting codes and provides a rigorous bound on the extractability of the entire file from any prover that successfully passes verification. However, these early schemes take into consideration data to be static and located at a single monolithic cloud server. To solve the problem of frequent updates of the data, Erway et al. (2015) and Shi et al. (2013) extended PDP/PoR to dynamic settings by introducing authenticated skip lists or rank-based authenticated dictionaries, enabling efficient verification of block-level modifications, insertions, and deletions. To further improve auditing efficiency, two main research directions have been explored. On the one hand, public auditing and batch verification mechanisms optimize auditing workflows and reduce management overhead (Shen et al., 2017; Guo et al., 2022; Liu et al., 2022). On the other hand, the integration of advanced cryptographic primitives and novel data structures significantly reduces computational overhead and enhances the practicality of PoS schemes (Anthoine et al., 2021; Tian et al., 2023; Lakshmi and T., 2025). Nevertheless, all the above schemes focus exclusively on the cloud side and do not account for the edge computing capabilities and data preprocessing requirements of fog nodes in fog–cloud collaborative architectures.

PoRep and PoST are designed to defend against replica-based Sybil attacks in decentralized storage by employing sealing mechanisms and time-dependent proofs to ensure authenticity and persistence. However, their data encoding procedures are computationally intensive (Damgård et al., 2019; Fisch, 2019), often taking hours and reducing access efficiency, and some rely on blockchain infrastructures (Zhang et al., 2021; Xu et al., 2024; Teng et al., 2025), which add computation and communication overhead. These factors make PoRep and PoST unsuitable for time-sensitive IoT applications requiring low-latency access. Furthermore, they remain confined to a two-tier architecture, auditing only the final storage holder and ignoring intermediate nodes such as fog layers.

2.2. Proof of storage in fog–cloud storage

To adapt proof of storage techniques to hierarchical architectures, several studies have extended traditional cloud-based auditing schemes to fog–cloud architectures. These works typically introduce fog nodes as intermediaries that assist in data preprocessing or audit execution, aiming to reduce computation and communication overhead on end devices.

Tian et al. (2019) were the first to propose a public auditing scheme for fog–cloud computing environments. Their scheme introduces a tag transformation mechanism between mobile sink nodes and fog nodes, enabling collaborative auditing between fog nodes and the cloud. In addition, zero-knowledge proofs are employed to protect the identity

privacy of data generators, preventing sensitive information leakage. Built on the discrete logarithm assumption, the scheme theoretically achieves both data privacy preservation and public auditability.

However, Zhang et al. (2023) later demonstrated through detailed security analysis that this scheme suffers from critical vulnerabilities. Specifically, a malicious cloud server can successfully deceive a Third-Party Auditor by launching data replacement and data deletion attacks while still generating seemingly valid audit proofs. As a result, the scheme fails to reliably guarantee data integrity and proof correctness. To address these weaknesses, the paper proposed an improved scheme that effectively mitigates the identified attacks and enhances overall security. While such enhancement is commendable, the modified scheme is still far from perfect. It especially does not consider data recoverability in fog–cloud architectures. When data gets corrupted because of a fault in fog nodes or cloud servers, the scheme does not help with effective data recovery. So, it limits the application of the scheme in complex and dynamic edge computing scenarios. Overall, research on storage auditing in fog–cloud architectures remains at an early exploratory stage.

3. Preliminaries

3.1. Bilinear map

Bilinear map (Boneh and Franklin, 2001) on elliptic curves are a crucial mathematical tool in cryptography. They pair two points on elliptic curves to generate a new value, and are widely applied in fields such as zero-knowledge proofs, short signatures, verifiable computation, and storage proofs.

Definition 1. Let G_1, G_2, and G_T be three cyclic groups of order p (a large prime), where G_1 and G_2 are additive groups on elliptic curves (which may be the same or different curves), and G_T is a multiplicative group (usually a group over a finite field). A bilinear map (or pairing), denoted as $e : G_1 \times G_2 \to G_T$, must satisfy the following three properties:

1. Bilinearity: For all $P \in G_1$, $Q \in G_2$, $a, b \in \mathbb{Z}_p$, $e(aP, bQ) = e(abP, Q) = e(bP, aQ) = e(P, Q)^{ab}$;
2. Non-degeneracy: There exist $P \in G_1$, $Q \in G_2$ such that $e(P, Q) \neq 1_{G_T}$;
3. Computability: There exists a polynomial-time algorithm that can efficiently compute $e(P, Q)$.

In this paper, bilinear maps are used to support succinct and publicly verifiable storage proofs. They enable the verification of polynomial commitments and aggregated audit responses through a small number of pairing operations, which makes the auditing process independent of the total data size. This property is particularly important in cloud-assisted IoT environments, where audits must remain efficient under frequent verification requests and large-scale data storage.

3.2. Polynomial commitments

Polynomial commitments (Kate et al., 2010) are used to create an encrypted digest C for a polynomial $f(X)$ of degree $\leq d$ over a finite field $\mathbb{Z}_p$, enabling subsequent proofs of the correctness of the evaluation $y = f(z)$ at any point z without revealing $f(X)$.

Definition 2. A polynomial commitment scheme is a 4-tuple of efficient algorithms:

$$PC = (Setup, Commit, Open, Check)$$

- $Setup(1^\kappa, d) \to pp$: Takes as input a security parameter κ and a maximum degree d, and generates public parameters pp.
- $Commit(pp, f(X)) \to (C, aux)$: Computes the corresponding commitment $C = f(r)$ at a point r, and outputs the commitment C and auxiliary information aux.
- $Open(pp, aux, z) \to (y, \pi)$: Selects a random number z, computes $y = f(z)$, and generates a proof π based on the value of y.
- $Check(pp, C, z, y, \pi) \to \{0, 1\}$: Verifies whether π indicates that y matches C.

In this paper, polynomial commitments are used to compactly represent Reed–Solomon encoded data blocks stored in the cloud. Each encoded block corresponds to an evaluation of a committed polynomial at a public index, allowing the cloud to generate concise responses to audit challenges. This design ensures data possession and consistency while significantly reducing communication and verification overhead, which aligns with the performance requirements of real-world cloud storage systems.

3.3. Pseudorandom function

A pseudorandom function (PRF) (Goldreich et al., 1984) is a deterministic function whose outputs are computationally indistinguishable from random values to any adversary without the secret key. PRFs are commonly used in security systems to generate unpredictable yet reproducible values with low computational overhead.

Definition 3. Let κ be the security parameter. A PRF family is a deterministic keyed function:

$$F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y},$$

where $\mathcal{K}$ is the key space, $\mathcal{X}$ is the input domain, and $\mathcal{Y}$ is the output range, all of which are implicitly parameterized by κ. The family F is said to be **secure** if for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$, the advantage

$$\mathrm{Adv}^{\mathrm{prf}}_{\mathcal{A},F}(\kappa) = \left|\Pr\left[\mathcal{A}^{F_k(\cdot)}(\kappa) = 1\right] - \Pr\left[\mathcal{A}^{f(\cdot)}(\kappa) = 1\right]\right|$$

is **negligible** in κ, where:

- $k \leftarrow \mathcal{K}$ is a uniformly random key;
- $f : \mathcal{X} \to \mathcal{Y}$ is a uniformly random function chosen from the set of all functions from $\mathcal{X}$ to $\mathcal{Y}$.

Here, $\mathcal{A}$ is given oracle access to either the PRF instance $F_k(\cdot)$ or a truly random function $f(\cdot)$, and cannot distinguish between the two worlds with non-negligible probability.

In our system, PRFs are employed to generate cryptographic watermarks for encoded data blocks at the fog layer. These watermarks are embedded into the stored blocks to enforce physical data storage by the cloud. Because the watermark values are indistinguishable from random data, the cloud cannot compress or selectively remove blocks without detection during audits. At the same time, the deterministic nature of PRFs allows authorized parties to efficiently verify watermark correctness without affecting data recovery.

4. System model

As shown in Fig. 1, our system involves four types of entities: IoT devices, fog nodes, cloud service providers (CSP), and a Third-party auditor (TPA).

- **IoT Devices**: IoT devices are resource-constrained sensors that continuously generate data. Due to limited storage and computational capabilities, they cannot retain data locally for long periods. Instead, the sensor performs preliminary authentication on the raw data within a fixed time window before forwarding the data to the fog node. After successful data outsourcing, IoT devices delete their local copies.

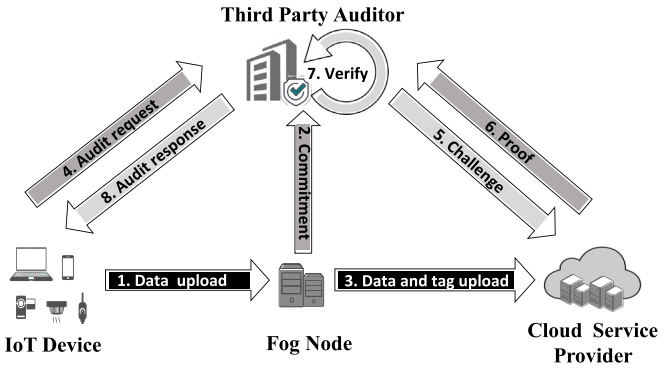

Fig. 1. Overview of our system model.

- **Fog Nodes**: Fog nodes are edge servers with moderate computational and storage capabilities, acting as intermediaries between IoT devices and the cloud. They perform data preprocessing operations, including encoding and generating cryptographic metadata required for storage auditing.
- **Cloud Service Provider (CSP)**: The CSP provides large-scale storage services and stores the outsourced data. They are required to respond to audit challenges by generating proofs of data possession.
- **Third-Party Auditor (TPA)**: The TPA is an independent and trusted entity that performs public auditing on behalf of data owners. It periodically issues audit challenges to the CSP and verifies the returned proofs without retrieving the original data.

4.1. Threat model

We consider four types of entities with distinct trust assumptions.

IoT Devices: IoT devices are assumed to be trusted data sources, consistent with prior work (Tian et al., 2019; Zhang et al., 2023). This assumption is supported by standard security mechanisms such as authentication, secure boot, trusted execution environments, and remote attestation (Kaur, 2024), which ensure device identity and execution integrity prior to data outsourcing.

Fog Nodes: Fog nodes are modeled as semi-honest or selfish entities that may deviate from prescribed processing steps to reduce computational or storage overhead. They are assumed not to collude with the cloud service provider. This assumption follows standard fog–cloud deployment models with independent administrative domains and distinct economic incentives (Huaranga-Junco et al., 2024).

Furthermore, for adversarial settings that consider potential collusion, existing works have explored dedicated solutions based on decentralized auditing and blockchain-based verification mechanisms (Chen et al., 2024; Seyedi et al., 2025) and adopted incremental verifiable computation to accelerate blockchain operations (Liu et al., 2025). These approaches address stronger threat models but typically incur additional communication and computation overhead. Nevertheless, they are complementary to our design and can serve as a potential direction for extending our framework to more stringent adversarial environments in future work.

CSP: The CSP is considered malicious. It may arbitrarily deviate from the protocol, including: (i) deleting parts of the outsourced data blocks, (ii) replacing valid blocks by stale or forged ones, and (iii) only storing a small portion of the data to reduce storage costs, but try to pass the auditing without actually storing the required data. The CSP has full access to the stored data, commitments, and proofs, but it is oblivious to the keys held by the fog nodes or the mobile receivers.

TPA: The TPA is assumed to be honest-but-curious. It performs the auditing protocol correctly, but it may try to learn more about outsourced data than the protocol allows. The TPA does not collude with the CSP or the fog nodes.

Overall, we assume trusted IoT devices, semi-honest fog nodes, a malicious CSP, and an honest-but-curious TPA, consistent with standard fog–cloud auditing models.

4.2. Security goals

Against the aforementioned threat model, the proposed scheme is designed to achieve the following security properties.

- **Data Integrity and Recoverability**. The TPA can verify that the cloud server correctly stores the outsourced data blocks corresponding to the committed polynomials. Provided that a sufficient number of coded blocks are available, the original IoT data can be fully reconstructed from the outsourced data. This property is critical for IoT applications where data is non-regenerative and must remain available for long-term analysis and compliance purposes.
- **Resilience to Deletion Attacks**. Without storing the entire set of coded data blocks, it is not possible for the CSP or any outside adversary to produce a valid and auditable response that passes the verification. This property still holds even when only some blocks are stored or when valid blocks are replaced with arbitrary values.
- **Fog Node Accountability**. Fog nodes cannot evade accountability for negligence in the data outsourcing process by remaining anonymous or forging information. If a fog node performs malicious operations during data preprocessing, encoding, or commitment generation, including block tampering, forged encoding metadata, and omission of critical procedures, the TPA can precisely identify the responsible entity and reconstruct the full violation process using the non-repudiable evidence chain preserved in the auditing protocol.

5. The proposed construction

5.1. Overview of the scheme

The proposed protocol consists of five algorithms: **Setup**, **KeyGen**, **Encode**, **ProofGen**, and **Verify**. An overview of the protocol is illustrated in Fig. 2

The protocol initiates with the **Setup**, which generates system public parameters, a common reference string (CRS), hash functions, and a PRF key. In the **KeyGen**, key pairs are produced for IoT devices and fog nodes, and PRF key shares are distributed via a fog node consortium using secure multi-party computation.

During the **Encode**, the IoT device generates authentication tags for raw data. After validating the tags, the fog node performs Reed–Solomon (RS) encoding (Con et al., 2024), embeds an incompressible watermark, generates a KZG commitment (Kate et al., 2010) and evaluation proof, and appends a digital signature. The associated data are then packaged and uploaded to the cloud server for persistent storage.

In the **ProofGen**, the TPA issues a random challenge, and the cloud server retrieves stored data to generate a cryptographic proof. In the **Verify**, the TPA audits fog node behavior and cloud storage integrity through four sequential checks: watermark integrity, block binding consistency, KZG proof verification, and fog node signature validation. If data recovery is required, the fog node requests a sufficient number of coded blocks from the cloud server; the raw IoT data are then reconstructed via watermark removal and RS decoding, followed by integrity verification.

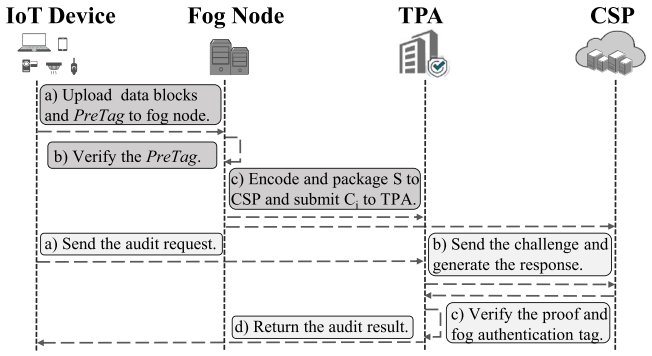

Fig. 2. Workflow of our proposed FastPoS.

5.2. The formal definition

Our scheme is defined by the following polynomial-time algorithms:

$\{Setup, KeyGen, Encode, ProofGen, Verify\}$.

- **Setup** $(1^\kappa, k, n) \to \mathsf{pp}$ Takes the security parameter κ, the encoding parameter k, and the system size parameter n as inputs, and outputs the public system parameters pp.
- **KeyGen** $(\mathsf{pp}) \to (spk, ssk, fpk, fsk, k_{\mathsf{prf}})$: Takes the public parameters pp as input, and outputs the key pairs (spk, ssk) and (fpk, fsk) for storage verification and fog-layer processing, respectively, together with a secret key k_{prf} for the pseudorandom function.
- **Encode** $(id_i, \{m_{i,\ell}\}_{\ell=1}^k, fsk_v) \to S$: Takes the data identifier id_i, the data blocks $\{m_{i,\ell}\}_{\ell=1}^k$, and the fog node secret key fsk_v as inputs, and outputs the processed storage data $S = (\{\tilde{c}_{i,j}, w_{i,j}\}_{j=1}^n, C_i, \{\pi_{i,j}\}_{j=1}^n, \sigma_i^{fog})$.
- **ProofGen** $(S, \mathsf{chal}) \to \mathsf{proof}$: Takes the stored data S and a verifier-generated challenge $\mathsf{chal} = (Q, \{\lambda_j\}_{j\in Q})$ as inputs, where $Q \subseteq \{1, \dots, n\}$ and $\{\lambda_j\} \subset \mathbb{Z}_p$, and outputs a storage proof $\mathsf{proof} = (\Phi, \Pi, \{\tilde{c}_{i,j}, w_{i,j}\}_{j\in Q})$.
- **Verify** $(\mathsf{pp}, \mathsf{chal}, \mathsf{proof}, C, fpk) \to (f, \mathsf{csp})$: Takes the public parameters pp, the challenge chal, the proof proof, the public commitment C, and the fog node public key fpk as inputs, and outputs (f, csp), where $f = 1$ if the fog node verification succeeds and $f = 0$ otherwise, and $\mathsf{csp} = 1$ if the cloud service provider verification succeeds and $\mathsf{csp} = 0$ otherwise.

5.3. The proposed scheme

Based on the formal definitions given above, this section elaborates on the implementation details of the proposed scheme as shown in Algorithm 1.

Setup. The setup algorithm takes as input a security parameter κ and coding parameters (k, n). It generates a bilinear group system $(\mathbb{G}, \mathbb{G}_T, p, g, e)$ of prime order p and samples a secret $\tau \xleftarrow{\$} \mathbb{Z}_p$ to construct the KZG common reference string:

$$\mathsf{CRS} = \{g, g^\tau, g^{\tau^2}, \dots, g^{\tau^d}\},$$

where $d \geq k-1$. It also fixes cryptographic hash functions H_1, H_2 and a pseudorandom function PRF. The public parameters pp include all the above system-wide parameters.

KeyGen. Each IoT device s_i samples a secret key $ssk_i \xleftarrow{\$} \mathbb{Z}_p$ and publishes $spk_i = g^{ssk_i}$. Each fog node f_v samples a secret key $fsk_v \xleftarrow{\$} \mathbb{Z}_p$ and publishes $fpk_v = g^{fsk_v}$. A system-wide PRF key k_{prf} is jointly generated by fog nodes using a threshold MPC protocol, such that no single fog node can reconstruct the full key.

Encode. This algorithm consists of two phases. The first phase is performed by IoT devices. They generate a preliminary authentication tag (PreTag) for data block $\{m_{i,\ell}\}_{\ell=1}^k$ as follows:

$$\sigma_{i,\ell}^{\mathsf{auth}} = H_1(id_i \| \ell \| m_{i,\ell})^{ssk_i}$$

Then, the fog node verifies each PreTag using the corresponding public key:

$$e(\sigma_{i,\ell}^{\mathsf{auth}}, g) \stackrel{?}{=} e(H_1(id_i, \ell, m_{i,\ell}), spk_i)$$

If any verification fails, the algorithm aborts.

The second phase is executed by the fog node. The fog node constructs a degree-$(k-1)$ polynomial $Q_i(x)$ such that $Q_i(\ell) = m_{i,\ell}$ for $\ell \in [1, k]$, and derives n encoded blocks:

$$c_{i,j} = Q_i(j), \quad j = 1, \dots, n.$$

For each encoded block, the fog node computes a watermark:

$$w_{i,j} = \mathsf{PRF}_{k_{\mathsf{prf}}}(id_i \| j \| fsk_v),$$

and embeds it into the block to obtain: $\tilde{c}_{i,j} \leftarrow c_{i,j} \oplus w_{i,j}$. The fog node then computes a KZG commitment:

$$C_i = g^{Q_i(\tau)}$$

and generates evaluation proofs $\pi_{i,j}$ for each $c_{i,j}$.

$$\pi_{i,j} = g^{(Q_i(\tau) - c_{i,j})/(\tau - j)}$$

In the end, the fog node signs the commitment to bind itself to the stored data:

$$\sigma_i^{\mathsf{fog}} = H_2(id_i, C_i)^{fsk_v}$$

The CSP stores the package S:

$$(\{\tilde{c}_{i,j}\}_{j=1}^n, \{w_{i,j}\}_{j=1}^n, C_i, \{\pi_{i,j}\}_{j=1}^n, \sigma_{i,\ell}^{\mathsf{auth}}, \sigma_i^{\mathsf{fog}})$$

ProofGen. The TPA samples a challenge set $\mathsf{chal} = (Q, \{\lambda_j\}_{j\in Q})$, where $Q \subseteq \{1, \dots, n\}$ and $\{\lambda_j\} \subset \mathbb{Z}_p$. Upon receiving the challenge, the CSP retrieves the corresponding stored blocks and computes:

$$\Phi = \sum_{j\in Q} \lambda_j \cdot c_{i,j}, \quad \Pi = \prod_{j\in Q} \pi_{i,j}^{\lambda_j},$$

where each $c_{i,j}$ is extracted from $\tilde{c}_{i,j}$ using $w_{i,j}$. The CSP returns $proof = (\Phi, \Pi, \{\tilde{c}_{i,j}, w_{i,j}\}_{j\in Q})$.

Verify. This algorithm is executed by the TPA upon receiving a storage proof from the cloud server. Its objective is twofold: (i) to enforce fog node accountability, and (ii) to verify cloud storage correctness. The verification procedure consists of the following steps.

The TPA first verifies whether the commitment associated with the outsourced data was correctly generated and authenticated by the fog node. Given the fog node's public key fpk_v and the commitment C_i, the TPA checks the fog signature:

$$e\left(\sigma_i^{\mathsf{fog}}, g\right) \stackrel{?}{=} e\left(H_2(C_i, id_i), fpk_v\right)$$

This step ensures that the fog node is cryptographically bound to the committed data and cannot repudiate its data processing behavior. If this verification fails, the audit is rejected, and the fog node is held accountable.

Then, the TPA verifies the correctness of the polynomial commitment proof by checking the KZG pairing equation:

$$e(C_i, g) \stackrel{?}{=} e(g^\Phi, g) \cdot e\left(\Pi,\ g^\tau \cdot \prod_{j\in Q} g^{-\lambda_j j}\right)$$

This step ensures that the values used to generate the proof are consistent with the polynomial committed by the fog node and that the cloud server possesses the corresponding encoded data blocks. If all checks pass, the audit is accepted.

Algorithm 1 FastPoS Scheme

▷ **Setup**
Inputs: κ
Outputs: pp
▷ **KeyGen**
Inputs: pp
Outputs: spk, ssk, fsk, fpk, k_{prf}
▷**Encode**
Inputs: $id_i, \{m_{i,\ell}\}_{\ell=1}^{k}, fsk_v$
Outputs: S
Generate $\sigma_{i,\ell}^{\mathsf{auth}}$
if $e(\sigma_{i,\ell}^{\mathsf{auth}}, g) = e(H_1(id_i, \ell, m_{i,\ell}), spk_i)$ **then**
  Generate $Q_i(x)$ such that $Q_i(\ell) = m_{i,\ell}$ for $\ell \in [1,k]$
  Generate n encoded block $c_{i,j}$
  $w_{i,j} \leftarrow \mathsf{PRF}_{k_{\mathsf{prf}}}(id_i \parallel j \parallel fsk_v)$
  $\tilde{c}_{i,j} \leftarrow c_{i,j} \oplus w_{i,j}$
  $C_i \leftarrow g^{Q_i(\tau)}$
  $\pi_{i,j} \leftarrow g^{(Q_i(\tau)-c_{i,j})/(\tau-j)}$
  $\sigma_i^{\mathsf{fog}} \leftarrow H_2(id_i, C_i)^{fsk_v}$
  $S \leftarrow (\{\tilde{c}_{i,j}\}_{j=1}^{n}, \{w_{i,j}\}_{j=1}^{n}, C_i, \{\pi_{i,j}\}_{j=1}^{n}, \sigma_i^{\mathsf{fog}})$
else
  verification fails
end if
▷**ProofGen**
Inputs: (S, chal)
Outputs: $proof$
Get Q and $\{\lambda_j\}_{j\in Q}$ from chal
$\Phi \leftarrow \sum_{j\in Q} \lambda_j \cdot c_{i,j}$
$\Pi \leftarrow \prod_{j\in Q} \pi_{i,j}^{\lambda_j}$,
$proof \leftarrow (\Phi, \Pi, \{\tilde{c}_{i,j}, w_{i,j}\}_{j\in Q})$
▷**Verify**
Inputs: $\mathsf{pp}, \mathsf{chal}, \mathsf{proof}, C, fpk$
Outputs: f,csp
$f, csp \leftarrow 1$
if $e\left(\sigma_i^{\mathsf{fog}}, g\right) \neq e\left(H_2\left(C_i, id_i\right), fpk_v\right)$ **then**
  $f \leftarrow 0$
else if $e(C_i, g) \neq e(g^{\Phi}, g) \cdot e\left(\Pi,\ g^{\tau} \cdot \prod_{j\in Q} g^{-\lambda_j j}\right)$ **then**
  $csp \leftarrow 0$
end if
return f, csp

Moreover, should partial data become corrupted and necessitate the recovery of original data, Algorithm 2 may be run to recover the affected data blocks.

First, the fog node initiates a data recovery request to the CSP. The request contains the unique identifier id_i of the target data and a set of coded block indices R of size k (where $R \subseteq [1,n]$, n is the total number of coded blocks, and k is the minimum number of blocks required for RS decoding):

$$\text{recov\_req} = \{id_i, R \subseteq [1,n], |R| = k\}$$

Upon receiving the request, the CSP retrieves and returns the coded block data corresponding to the index set R from storage, including the PreTag and $\{\tilde{c}_{i,j}, w_{i,j}\}_{j\in R}$. Then, the fog node first extracts the original coded blocks $c_{\{i,j\}}$ from the watermarked coded blocks via an XOR operation:

$$c_{i,j} = \tilde{c}_{i,j} \oplus w_{i,j}$$

After that, it performs RS decoding on the extracted k original coded blocks using Lagrange interpolation to recover the corresponding data polynomial $Q_i(x)$. Next, the recovered polynomial $Q_i(x)$ is evaluated at the specified point ℓ to obtain the original data block $m_{\{i,\ell\}}$. Finally,

Algorithm 2 Data Recovery

Inputs: id_i, C_i, k_{prf}, fsk_v
Output: $\{m_{i,\ell}\}_{\ell=1}^{k}$ or $\perp$
Select a recovery index set $R \subseteq \{1,\ldots,n\}$ with $|R| = k$
Send recov_req = $\{id_i, S\}$ to CSP
CSP returns $\{\tilde{c}_{i,j}, w_{i,j}\}_{j\in R}$ and PreTag_i
for each $j \in S$ **do**
  if $w_{i,j} \neq \mathsf{PRF}_{k_{\mathsf{prf}}}(id_i \parallel j \parallel fsk_v)$ **then**
    return $\perp$
  end if
  $c_{i,j} \leftarrow \tilde{c}_{i,j} \oplus w_{i,j}$
end for
Perform RS decoding to reconstruct $Q_i(x)$
for $\ell = 1$ **to** k **do**
  $m_{i,\ell} \leftarrow Q_i(\ell)$
end for
$C_i' \leftarrow g^{Q_i(\tau)}$
if $C_i' \neq C_i$ **then**
  return $\perp$
end if
$\mathrm{PreTag}_i' \leftarrow H_1(id_i \parallel \ell \parallel m_{i,\ell})^{ssk_i}$
if $\mathrm{PreTag}_i' \neq \mathrm{PreTag}_i$ **then**
  return $\perp$
end if
return $\{m_{i,\ell}\}_{\ell=1}^{k}$

the KZG commitment C_i corresponding to the data is recomputed and compared with the pre-stored commitment to verify the integrity of the recovered data. In addition, the PreTag can be recalculated using the recovered original data block $m_{\{i,\ell\}}$ and cross-checked against the PreTag stored in the cloud, thus verifying that fog nodes have not arbitrarily discarded or tampered with the data.

Discussion. It is worth noting that, although the system is described in a fog–cloud setting, it is not limited to fog-specific infrastructures. The design is based on a general intermediate-layer computation model, where edge nodes with comparable preprocessing and data forwarding capabilities can directly execute the proposed protocol without any modification to the underlying cryptographic mechanisms.

6. Security analysis

We analyze correctness, integrity against deletion and replacement attacks, and fog node accountability. All adversaries are probabilistic polynomial-time (PPT).

6.1. Cryptographic assumptions

We rely on the following standard cryptographic assumptions.

- q-**Strong Diffie–Hellman** (q-**SDH**) **assumption.** Given a tuple $(g, g^{\tau}, \ldots, g^{\tau^q})$ for randomly chosen $\tau \in \mathbb{Z}_p$, it is computationally infeasible to output a pair $(c, g^{1/(\tau+c)})$.
- **Discrete Logarithm (DL) assumption.** Given $g^x \in \mathbb{G}$ for random $x \in \mathbb{Z}_p$, it is computationally infeasible to recover x.
- **EUF-CMA signature security.** The digital signature scheme is existentially unforgeable under adaptive chosen-message attacks.

6.2. Correctness

Theorem 1 (*Correctness*)**.** *If all entities follow the protocol honestly, then the verification algorithm accepts.*

Proof. All arithmetic operations involving the Lagrange coefficients λ_j are performed over the scalar field $\mathbb{Z}_p$ associated with the bilinear groups $(\mathbb{G}, \mathbb{G}_T)$. We verify correctness by showing that each pairing equation holds under honest execution.

(i) Authentication equation.

$$e(\sigma_{i,\ell}^{\text{auth}}, g) = e(H_1(id_i, \ell, m_{i,\ell}), spk_i) \tag{1}$$

Since $\sigma_{i,\ell}^{\text{auth}} = H_1(id_i, \ell, m_{i,\ell})^{ssk_i}, spk_i = g^{ssk_i}$, by bilinearity of the pairing map, we have

$$LHS = e(H_1(id_i, \ell, m_{i,\ell})^{ssk_i}, g) = e(H_1(id_i, \ell, m_{i,\ell}), g)^{ssk_i},$$
$$RHS = e(H_1(id_i, \ell, m_{i,\ell}), g^{ssk_i}) = e(H_1(id_i, \ell, m_{i,\ell}), g)^{ssk_i}.$$

Therefore, Equation (1) holds.

(ii) Fog signature equation.

$$e(\sigma_i^{\text{fog}}, g) = e(H_2(C_i, id_i), fpk_v) \tag{2}$$

Similar to the proof of Eq. (1), by substituting $\sigma_i^{\text{fog}} = H_2(C_i, id_i)^{fsk_v}$, $fpk_v = g^{fsk_v}$, and applying the bilinearity of the pairing map, we can readily obtain that Eq. (2) holds.

(iii) Polynomial commitment equation.

$$e(C_i, g) = e(g^{\Phi}, g) \cdot e\left(\Pi, g^{\tau} \cdot \prod_{j\in Q} g^{-\lambda_j j}\right) \tag{3}$$

Since $C_i = g^{Q_i(\tau)}, c_{i,j} = Q_i(j)$, and for each challenge index $j \in Q$, the corresponding witness is $\pi_{i,j} = g^{(Q_i(\tau)-c_{i,j})/(\tau-j)}$,

we have

$$\begin{aligned} e(g^{c_{i,j}}, g) \cdot e(\pi_{i,j}, g^{\tau-j}) &= e(g^{c_{i,j}}, g) \cdot e(g^{(Q_i(\tau)-c_{i,j})/(\tau-j)}, g^{\tau-j}) \\ &= e(g^{c_{i,j}}, g) \cdot e(g^{Q_i(\tau)-c_{i,j}}, g) \\ &= e(g^{Q_i(\tau)}, g) \\ &= e(C_i, g) \end{aligned}$$

Thus, the single-point verification is complete.

Next, for each challenge index $j \in Q$, raise both sides to exponent λ_j and multiply all resulting equations:

$$\prod_{j\in Q} e(C_i, g)^{\lambda_j} = \prod_{j\in Q} e(g^{\lambda_j c_{i,j}}, g) \cdot \prod_{j\in Q} e(\pi_{i,j}^{\lambda_j}, g^{\tau-j}) \tag{4}$$

Because the Lagrange coefficients satisfy $\sum_{j\in Q} \lambda_j = 1$, for Eq. (4), we have

$$LHS = e(C_i, g)^{\sum_{j\in Q} \lambda_j} = e(C_i, g).$$

By the bilinearity and multiplicative homomorphism of the pairing map, together with the definitions of the aggregated value Φ and the aggregated witness Π, we have

$$\prod_{j\in Q} e(g^{\lambda_j c_{i,j}}, g) = e(g^{\Phi}, g) \tag{5}$$

Similarly,

$$\prod_{j\in Q} e(\pi_{i,j}^{\lambda_j}, g^{\tau-j}) = e\left(\Pi, \prod_{j\in Q} g^{\lambda_j(\tau-j)}\right) \tag{6}$$

Expanding the exponent gives

$$\sum_{j\in Q} \lambda_j(\tau - j) = \tau - \sum_{j\in Q} \lambda_j j \tag{7}$$

Therefore,

$$\prod_{j\in Q} g^{\lambda_j(\tau-j)} = g^{\tau} \cdot \prod_{j\in Q} g^{-\lambda_j j}. \tag{8}$$

Substituting these expressions into Equation (4) yields the right-hand side, we have

$$RHS = e(g^{\Phi}, g) \cdot e\left(\Pi, g^{\tau} \cdot \prod_{j\in Q} g^{-\lambda_j j}\right).$$

Thus, Equation (3) holds. □

6.3. Resistance to deletion and replacement attacks

Theorem 2 (*Integrity against Deletion and Replacement*)**.** *Any PPT adversary that outputs an accepting proof without storing the correct dataset, succeeds only with negligible probability in the security parameter* κ.

Proof. We analyze deletion attacks and replacement attacks separately.

(i) Detection of Data Deletion. Let $D \subseteq [n]$ denote the set of deleted indices, where $|D| = d$. During each audit, the verifier samples a challenge set $Q \subseteq [n]$ uniformly at random without replacement, where $|Q| = t$.

Define the success event of the adversarial cloud as

$$\text{Succ} := \{Q \cap D = \emptyset\}.$$

That is, the adversary succeeds only if none of the deleted indices are selected in the challenge set. The probability of this event is

$$\Pr[\text{Succ}] = \frac{\binom{n-d}{t}}{\binom{n}{t}} = \prod_{i=0}^{t-1} \frac{n-d-i}{n-i}.$$

For all $i \in \{0, \ldots, t-1\}$, we have

$$\frac{n-d-i}{n-i} \le \frac{n-d}{n}.$$

Therefore,

$$\Pr[\text{Succ}] \le \left(1 - \frac{d}{n}\right)^t.$$

After κ independent verification rounds, the probability that the deletion remains undetected becomes

$$\Pr[\text{Succ}_\kappa] \le \left(1 - \frac{d}{n}\right)^{t\kappa}.$$

Since $t\kappa$ grows linearly in the security parameter, and d/n is a positive constant, this probability is negligible.

(ii) Replacement attacks.

Assume there exists a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ that replaces at least one data block and still produces an accepting proof with non-negligible probability.

Let the adversary output forged values $\{\tilde{c}_{i,j}\}_{j\in Q}$ with at least one $\tilde{c}_{i,j} \neq c_{i,j}$. To pass verification, the adversary must construct an aggregate witness $\tilde{\Pi}$ satisfying

$$e(C_i, g) = e(g^{\tilde{\Phi}}, g) \cdot e\left(\tilde{\Pi}, g^{\tau} \cdot \prod_{j\in Q} g^{-\lambda_j j}\right) \tag{9}$$

where

$$\tilde{\Phi} = \sum_{j\in Q} \lambda_j \tilde{c}_{i,j}.$$

Since the commitment $C_i = g^{Q_i(\tau)}$ is publicly known, constructing such a witness requires computing

$$\tilde{\Pi} = g^{\frac{Q_i(\tau)-\tilde{\Phi}}{\tau-\sum_{j\in Q}\lambda_j j}}.$$

However, the adversary only possesses the group element C_i and does not know the exponent $Q_i(\tau)$. Therefore, producing a valid forged witness implies computing the discrete logarithm of C_i in $\mathbb{G}$.

Formally, any PPT adversary that succeeds in a replacement attack with non-negligible probability can be used to construct a reduction that solves the Discrete Logarithm (DL) problem with non-negligible probability, contradicting the DL assumption.

Thus, replacement attacks succeed only with probability $\text{negl}(\kappa)$. □

6.4. Fog node accountability

Theorem 3 (*Fog Node Accountability*). *For any probabilistic polynomial-time (PPT) adversary controlling a fog node, any deviation from the prescribed preprocessing, encoding, or commitment generation procedures is detected and can be attributed to the responsible fog node with overwhelming probability.*

Proof. The accountability of the fog node follows from three cryptographic properties: the binding nature of polynomial commitments, the verifiability of encoding operations through pairing-based proofs, and the non-repudiation guarantee provided by digital signatures.

(i) Commitment binding.

During the initialization phase, the fog node generates the commitment

$$C_i = g^{Q_i(\tau)}$$

for the encoded data polynomial $Q_i(x)$.

By the binding property of KZG polynomial commitments, it is computationally infeasible for a PPT adversary to produce two distinct polynomials $Q_i(x)$ and $\tilde{Q}_i(x)$ such that

$$C_i = g^{Q_i(\tau)} = g^{\tilde{Q}_i(\tau)}.$$

Formally, under the q-SDH assumption,

$$\Pr\left[Q_i \neq \tilde{Q}_i \wedge g^{Q_i(\tau)} = g^{\tilde{Q}_i(\tau)}\right] \leq \mathrm{negl}(\kappa).$$

Therefore, the commitment uniquely binds the fog node to its original encoding.

(ii) Encoding correctness.

Suppose the fog node produces incorrectly encoded values $c_{i,j} \neq Q_i(j)$ for d indices.

Such inconsistencies are detected whenever a corrupted index is selected during verification. Let Q denote the challenge set with size t.

The probability that all corrupted indices remain undetected after κ independent verification rounds satisfies

$$\Pr[\text{undetected}] \leq \left(1 - \frac{d}{n}\right)^{t\kappa},$$

which is negligible in the security parameter.

(iii) Non-repudiation via signatures.

The fog node signs the commitment C_i, the encoded values $\{c_{i,j}\}$, and the witness set $\{\pi_{i,j}\}$.

Under the existential unforgeability under adaptive chosen-message attacks (EUF-CMA) assumption, no PPT adversary can produce a valid signature on a message that was not previously signed. Therefore, the fog node cannot deny its operations or attribute them to another entity.

Combining the above arguments, any deviation from the prescribed protocol is detected with overwhelming probability and can be cryptographically attributed to the responsible fog node. □

7. Performance analysis

In this section, we present the theoretical computational costs of the proposed scheme, and then compare its actual performance with relevant schemes through experiments.

7.1. Theoretical analysis

We list all necessary notations for theoretical analysis in Table 2 and compare the computational costs in the tag generation phase and proof verification phase, as well as the communication costs, between the proposed scheme and representative fog–cloud auditing schemes in Table 3. The comparison focuses on dominant cryptographic operations, including exponentiations, bilinear pairings, and hash evaluations, while omitting lightweight arithmetic operations whose costs are negligible in practice.

Table 2
Notations of theoretical analysis.

Notation	Description
M	Multiplication operation over group G
E	Exponentiation operation over group G
P	Pairing operation over groups G and G_T
H	Hash operation
c	Number of challenged data blocks
t	Number of challenge rounds
$\|G\|$	Bit length of an element in group G
$\|\mathbb{Z}_p\|$	Bit length of an element in finite field $\mathbb{Z}_p$

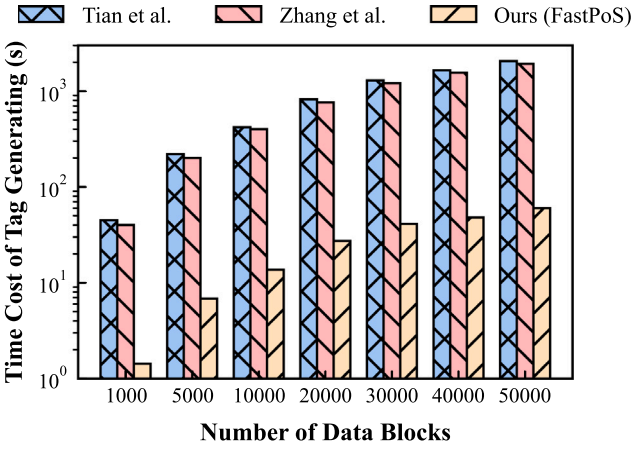

Fig. 3. Time cost of tag generation under different numbers of blocks.

At the tag generation phase, the computational cost of FastPoS is comparable to existing tag-based solutions. Similar to Zhang et al. (2023), our scheme requires a constant number of exponentiations and hash operations to generate a polynomial commitment for each data object. However, unlike traditional per-block tag generation, the commitment in FastPoS binds the whole coded data polynomial at once, avoiding linear growth with respect to the number of data blocks.

The proof verification phase achieves greater efficiency gains. In Tian et al. (2019) and Zhang et al. (2023), the verifier needs to process a number of pairing and exponentiation operations that grow linearly with the challenge size, since each challenged block contributes independently to the verification equation. In contrast, FastPoS supports proof aggregation through polynomial commitments, enabling the verifier to validate the aggregated proof using only a constant number of pairing operations, independent of the number of challenged blocks. Although the number of exponentiation operations grows linearly with the number of challenged blocks, the substantial reduction in pairing operations leads to significant efficiency improvements. This property substantially reduces the verification overhead, making it particularly suitable for large-scale and high-frequency auditing scenarios.

In terms of communication cost, the proposed scheme further outperforms existing approaches. Traditional tag-based auditing schemes require the cloud server to return per-block responses for each challenged block, leading to communication overhead linear in the challenge size. By contrast, ours only requires transmitting a constant-size aggregated proof and a small number of field elements, resulting in communication complexity independent of the challenge set size. This constant-size proof property makes the proposed scheme particularly suitable for bandwidth-constrained fog–cloud environments and latency-sensitive IoT applications.

Overall, our scheme achieves significantly lower verification and communication costs while maintaining comparable preprocessing overhead, thereby offering a more scalable and efficient auditing solution for fog–cloud storage systems.

Table 3
Theoretical comparison of computational and communication costs.

Scheme	Tag generation	Proof verification	Communication cost
(Tian et al., 2019)	$3 \cdot E + 1 \cdot M + 1 \cdot H$	$2t \cdot E + t \cdot P + 1 \cdot H$	$t \cdot \|G\| + t \cdot \|\mathbb{Z}_p\|$
(Zhang et al., 2023)	$2 \cdot E + 1 \cdot M + 1 \cdot H$	$(t + c) \cdot E + t \cdot P + 1 \cdot H$	$t \cdot \|G\| + t \cdot \|\mathbb{Z}_p\|$
FastPoS (Ours)	$2 \cdot E + 1 \cdot M + 1 \cdot H$	$t \cdot E + 1 \cdot P + 1 \cdot H$	$1 \cdot \|G\| + t \cdot \|\mathbb{Z}_p\|$

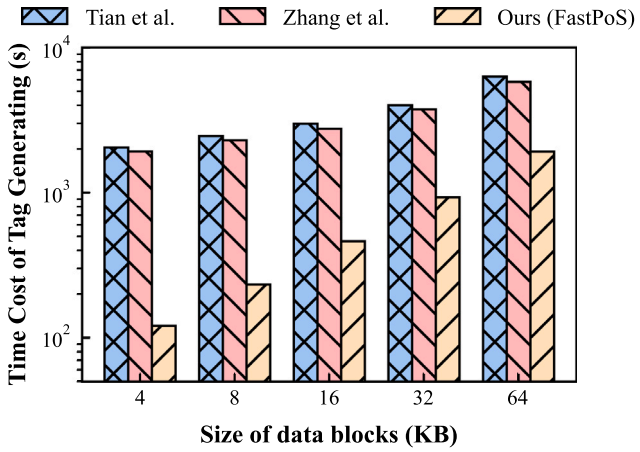

Fig. 4. Time cost of tag generation under different data block sizes.

7.2. Experimental evaluation

In this section, we conduct an experimental evaluation of the proposed scheme and compare it with Tian et al.'s scheme (Tian et al., 2019) and Zhang et al.'s scheme (Zhang et al., 2023).

Experimental setup. All experiments were conducted on a Windows 11 system equipped with an Intel Core i7-14700 CPU at 2.10 GHz and 16 GB of RAM. The proposed scheme was implemented using the ckzg library (version 2.1.1), which provides a production-grade implementation of KZG polynomial commitments consistent with the Ethereum specification. The cryptographic parameters, including the trusted setup and common reference string, were instantiated using the official library configuration. To reflect lightweight edge–fog computing scenarios, we adopt short-code Reed–Solomon (RS) coding parameters. Unless otherwise specified, the RS configuration is fixed to $(k, r) = (10, 4)$ in the performance evaluation, where k is the number of data shards and r is the number of parity shards. Each experiment was repeated 15 times, and the reported results correspond to the average value.

Experimental data. Considering the continuous data appending behavior in practical IoT systems, we use randomly generated byte streams to emulate incremental data generated by multiple IoT devices within a time window. We further vary the data block size and the number of data blocks to evaluate system scalability under different workloads. We vary the data block size and the number of data blocks to evaluate system scalability under different workloads.

In this setting, the cost of appending new data is modeled as an independent storage proof instance, i.e., each newly arrived batch incurs a fresh proof generation cost equivalent to re-running the storage proof process. This abstraction simplifies the evaluation of dynamic workloads in IoT environments. More efficient mechanisms for incremental proof updating and amortized verification are left as future work.

The time cost of tag generation. We first evaluate the time cost of generating authentication tags with a fixed data block size (4 KB) while varying the number of data blocks from 1000 to 50,000. As shown in Fig. 3, FastPoS consistently outperforms the competing schemes under all experimental scales, and significant differences can be observed among the three schemes in terms of both absolute overhead and growth rate. The time cost of Tian et al. and Zhang et al. increases almost linearly with the number of blocks, rising from approximately 100 s to over 1000 s. In stark contrast, the time cost of FastPoS remains well below 100 s across all tested scales and exhibits a much slower growth rate. When the number of data blocks is 1000, the tag generation time of FastPoS is only 1.4 s, whereas those of Zhang et al.'s scheme and Tian et al.'s scheme are 40 s and 45 s, respectively—approximately 28.(d)× and 31.(d)× that of our scheme. As the data scale increases, the gap widens further. At 50,000 data blocks, the computational overheads of the two compared schemes stand at approximately 31.(d)× and 34.(d)× that of FastPoS, respectively. This proves our scheme retains significant performance advantages in large-scale data scenarios.

Furthermore, we assess the impact of data block size on tag generation latency, fixing the number of blocks at 50,000. As illustrated in Fig. 4, all three schemes exhibit increased tag generation time as the block size scales from 4 KB to 64 KB. Consistent with the prior experiment, FastPoS retains its performance superiority across all tested configurations. While Tian et al.'s and Zhang et al.'s schemes incur a relatively high and stable latency ranging from 2000 ms to 8000 ms, FastPoS's latency remains drastically lower, staying below 2000 ms even at the maximum block size of 64 KB. When the block size is 4 KB, the tag generation time of our scheme is 121 s. In contrast, Zhang et al.'s scheme and Tian et al.'s scheme take 1920 s and 2050 s for tag generation. Their respective computational overheads are about 15.(d)× and 16.(d)× that of our scheme. Although the relative ratio decreases slightly in large-block scenarios, the proposed scheme still significantly outperforms the compared schemes in both absolute time and overall efficiency. This further validates the efficiency of our polynomial commitment-based design.

The time cost of proof verification. We evaluate the time overhead of proof verification for the three schemes under different detection probability settings. The detection probability p and challenge size C satisfy the relationship $1 - (1 - \rho)^C \geq p$, where ρ denotes the data error rate and $1 - \rho$ represents the detection probability.

As shown in Fig. 5, the verification time of all three schemes increases monotonically with the detection probability. This is because a higher detection probability implies a larger challenge size, which requires verifying more proof elements related to the challenged data blocks. This trend is fully consistent with the theoretical analysis. In terms of absolute time overhead, the proposed scheme exhibits significant advantages under all detection probability settings. When the detection probability is 60%, the verification time of the proposed scheme is only 77 ms, whereas those of Zhang et al.'s scheme and Tian et al.'s scheme reach 5000 ms and 4800 ms, respectively—approximately 65.(d)× and 62.(d)× that of the proposed scheme. When the detection probability is increased to 99%, the proposed scheme can still complete verification within sub-second time, while the compared schemes require more than ten seconds, making it difficult to meet the requirements of high-frequency auditing or real-time monitoring scenarios.

To further examine scalability, we analyze the verification time of the proposed scheme under different data block sizes (4–64 KB) and detection probabilities (60%–99%), as shown in Fig. 6. The results show that, for a fixed block size, the verification time increases gradually with the detection probability. This confirms that the dominant factor affecting verification cost is the challenge scale, and that the computational complexity grows in a predictable and stable manner.

Moreover, for a fixed detection probability, increasing the block size has little effect on validation time, especially when the detection probability is small, where the validation time difference between different block sizes is only a few milliseconds. This indicates that the verification procedure mainly depends on polynomial commitment

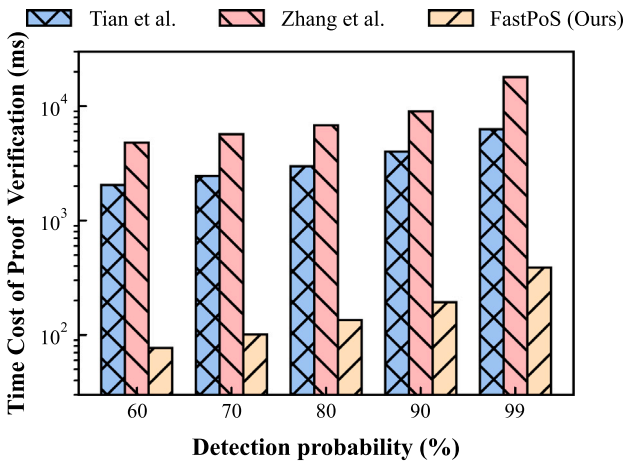

Fig. 5. Time cost of proof verification under different detection probabilities.

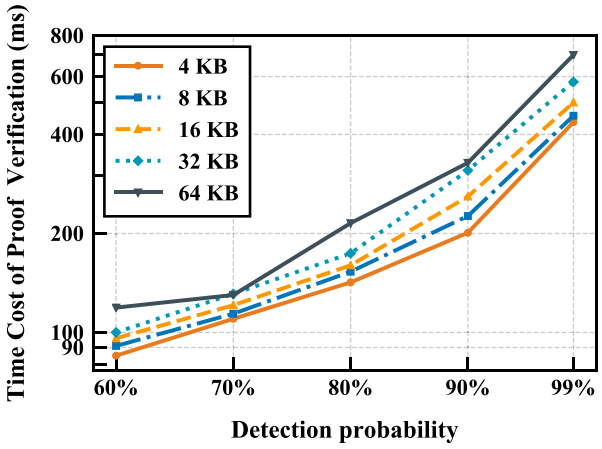

Fig. 6. Time cost of proof verification under different data block sizes and detection probabilities.

verification and algebraic operations, rather than direct processing of raw data content. Consequently, the proposed scheme exhibits low sensitivity to block size variations, which is desirable in practical IoT environments where data granularity may differ across applications.

Data recoverability analysis. We test the maximum tolerable missing blocks ratio across diverse settings with varying data block sizes and data block numbers. All missing fragments are randomly selected during experiments to realistically simulate random data loss and fragment corruption in practical edge and IoT deployment environments. In our setting, when the missing ratio is within 40%, the system enables accurate data reconstruction steadily under all experimental settings. In contrast, once the missing ratio exceeds 40%, the fundamental decoding constraints of Reed–Solomon codes can no longer be satisfied, inevitably resulting in the failure of data reconstruction. This threshold-based recovery characteristic is fully consistent with the theoretical decoding mechanism of RS erasure coding, which further validates the reliability and feasible data recoverability of our designed scheme.

In summary, the experimental evaluation validates that FastPoS achieves the performance characteristics essential for practical deployment in large-scale IoT environments. The combination of lightweight tag generation, sub-second verification latency, and constant-size proofs enables high-frequency auditing without imposing prohibitive computational or communication overhead. These properties position FastPoS as a scalable solution that effectively bridges the gap between strong security guarantees and real-time operational requirements in fog–cloud storage systems.

8. Conclusion

This paper presents an efficient PoS scheme based on KZG polynomial commitment, which is suitable for large-scale fog–cloud architectures. By redesigning the tag generation and verification procedures, the scheme achieves strong data integrity assurance while significantly reducing computational overhead. Experimental results show that the proposed scheme consistently outperforms existing schemes by Zhang et al. (2023) and Tian et al. (2019) and maintains sub-second proof verification time even under 99% detection probability. These results indicate that the proposed scheme effectively addresses the scalability and efficiency limitations of prior storage auditing approaches.

CRediT authorship contribution statement

Yuting An: Writing – original draft, Methodology, Formal analysis. **Helei Cui:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Conceptualization. **Hao Zeng:** Validation, Methodology. **Xiaoning Liu:** Validation, Formal analysis. **Bin Guo:** Resources, Conceptualization. **Zhiwen Yu:** Resources, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. U22B2022, 62372383).

Data availability

Data will be made available on request.

References

Anthoine, G., Dumas, J.G., de Jonghe, M., Maignan, A., Pernet, C., Hanling, M., Roche, D.S., 2021. Dynamic proofs of retrievability with low server storage. In: 30th USENIX Security Symposium. USENIX Security 21, USENIX Association, pp. 537–554.

Ateniese, R., Curtmola, R., Herring, J., Kissner, L., Peterson, Z., Song, D., 2007. Provable data possession at untrusted stores. In: Proceedings of the 14th ACM Conference on Computer and Communications Security. CCS'07, New York, NY, USA, pp. 598–609.

Boneh, D., Franklin, M., 2001. Identity-based encryption from the weil pairing. In: Advances in Cryptology — CRYPTO 2001. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 213–229.

Chen, X., Cheng, Q., Yang, W., Luo, X., 2024. An anonymous authentication and secure data transmission scheme for the internet of things based on blockchain. Front. Comput. Sci. 18 (3).

Con, R., Shutty, N., Tamo, I., Wootters, M., 2024. Repairing reed-solomon codes over prime fields via exponential sums. IEEE Trans. Inform. Theory 70 (12), 8587–8594.

Damgård, I., Ganesh, C., Orlandi, C., 2019. Proofs of replicated storage without timing assumptions. In: Advances in Cryptology – CRYPTO 2019. Cham, pp. 355–380.

Erway, C.C., Küpçü, A., Papamanthou, C., Tamassia, R., 2015. Dynamic provable data possession. ACM Trans. Inf. Syst. Secur. 17 (4).

Fisch, B., 2019. Tight proofs of space and replication. In: Advances in Cryptology – EUROCRYPT 2019. Springer International Publishing, Cham, pp. 324–348.

Goldreich, O., Goldwasser, S., Micali, S., 1984. How to construct random functions. In: 25th Annual Symposium OnFoundations of Computer Science, 1984. pp. 464–479.

Guo, W., Qin, S., Gao, F., Zhang, H., Li, W., Jin, Z., Wen, Q., 2022. Dynamic proof of data possession and replication with tree sharing and batch verification in the cloud. IEEE Trans. Serv. Comput. 15 (4), 1813–1824.

Huaranga-Junco, E., González-Gerpe, S., Castillo-Cara, M., Cimmino, A., García-Castro, R., 2024. From cloud and fog computing to federated-fog computing: A comparative analysis of computational resources in real-time IoT applications based on semantic interoperability. Future Gener. Comput. Syst. 159, 134–150.

Juels, A., Kaliski, B.S., 2007. Pors: proofs of retrievability for large files. In: Proceedings of the 14th ACM Conference on Computer and Communications Security. CCS'07, New York, NY, USA, pp. 584–597.

Kaaniche, N., Laurent, M., 2017. Data security and privacy preservation in cloud storage environments based on cryptographic mechanisms. Comput. Commun. 111, 120–141.
Kate, A., Zaverucha, G.M., Goldberg, I., 2010. Constant-size commitments to polynomials and their applications. In: Advances in Cryptology - ASIACRYPT 2010. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 177–194.
Kaur, N., 2024. A systematic review on security aspects of fog computing environment: Challenges, solutions and future directions. Comput. Sci. Rev. 54, 100688.
Lakshmi, V.R.V., T., G.K., 2025. Enhancing data integrity in opportunistic mobile social network: Leveraging Berkle tree and secure data routing against attacks. Comput. Secur. 148, 104133.
Le, T., Huang, P., Yavuz, A.A., Shi, E., Hoang, T., 2023. Efficient dynamic proof of retrievability for cold storage. In: Proceedings of the 30th Network and Distributed System Security Symposium. NDSS 2023, San Diego, CA, USA, pp. 1–18.
Liu, Z., Ren, L., Li, R., Liu, Q., Zhao, Y., 2022. ID-based sanitizable signature data integrity auditing scheme with privacy-preserving. Comput. Secur. 121, 102858.
Liu, L., Wei, P., Liu, S., Wang, Z., Hu, D., Kou, Z., 2025. Scalable batch verification of ECDSA for blockchain using IVC. Front. Comput. Sci. 20 (4).
Moran, T., Orlov, I., 2019. Simple proofs of space-time and rational proofs of storage. In: Advances in Cryptology – CRYPTO 2019. Cham, pp. 381–409.
Seyedi, Z., Rahmati, F., Ali, M., Liu, X., 2025. A fully decentralized auditing approach for edge computing: A game-theoretic perspective. IEEE Trans. Dependable Secur. Comput. 22 (6), 6121–6132.
Shen, J., Shen, J., Chen, X., Huang, X., Susilo, W., 2017. An efficient public auditing protocol with novel dynamic structure for cloud data. IEEE Trans. Inf. Forensics Secur. 12 (10), 2402–2415.
Shi, E., Stefanov, E., Papamanthou, C., 2013. Practical dynamic proofs of retrievability. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security. CCS '13, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450324779, pp. 325—-336.
Teng, Y., Lv, J., Wang, Z., Gao, Y., Dong, W., 2025. TimeChain: A secure and decentralized off-chain storage system for IoT time series data. In: Proceedings of the ACM on Web Conference 2025. WWW '25, Association for Computing Machinery, New York, NY, USA, pp. 3651—-3659.
Tian, H., Nan, F., Chang, C.C., Huang, Y., Lu, J., Du, Y., 2019. Privacy-preserving public auditing for secure data storage in fog-to-cloud computing. J. Netw. Comput. Appl. 127, 59–69.
Tian, M., Zhang, Y., Zhu, Y., Wang, L., Xiang, Y., 2023. DIVRS: Data integrity verification based on ring signature in cloud storage. Comput. Secur. 124, 103002.
Xu, M., Zhang, J., Guo, H., Cheng, X., Yu, D., Hu, Q., Li, Y., Wu, Y., 2024. FileDES: A secure, scalable and succinct decentralized encrypted storage network. In: IEEE INFOCOM 2024 - IEEE Conference on Computer Communications. pp. 261–270.
Zhang, W., Jiao, H., Yan, Z., Wang, X., Khan, M.K., 2023. Security analysis and improvement of a public auditing scheme for secure data storage in fog-to-cloud computing. Comput. Secur. 125, 103019.
Zhang, Y., Xu, C., Lin, X., Shen, X., 2021. Blockchain-based public integrity verification for cloud storage against procrastinating auditors. IEEE Trans. Cloud Comput. 9 (3), 923–937.

Yuting An received the B.S. degree in information security from Hainan University, Hainan, China in 2024. She is currently working toward the M.S degree with the Northwestern Polytechnical University. Her research interests include cloud security, data security and decentralized cloud storage.

Helei Cui received the Ph.D. degree in Computer Science from the City University of Hong Kong, in 2018. He is currently a Professor with the School of Computer Science, Northwestern Polytechnical University, China. His research interests include Trustworthy Crowd Computing, Privacy-Preserving Computing, Edge Intelligence, Decentralized Cloud Storage, etc.

Hao Zeng received his B.S. degree in computer science and technology from Northwestern Polytechnical University, Xi'an, China in 2023. He is currently working toward the Ph.D. degree with the Northwestern Polytechnical University. His research interests include blockchain, data security, and cloud security.

Xiaoning Liu is a Senior Lecturer and an ARC DECRA Fellow at the School of Computing Technologies, RMIT University, Australia. Her research interests include secure computation, machine learning security and privacy. She is the recipient of the Best Paper Award of ESORICS 2021, the RMIT HDR Research Prize 2023.

Bin Guo received the Ph.D. degree in computer science from Keio University, Minato, Japan, in 2009. He was a Postdoctoral Researcher with the Institut TELECOM Sud-Paris, Essonne, France. He is currently a Professor with Northwestern Polytechnical University, Xi'an, China. His research interests include ubiquitous computing, mobile crowd sensing, and HCI.

Zhiwen Yu received the M.E. and Ph.D. degrees from Northwestern Polytechnical University, Xi'an, China in 2003 and 2005, respectively. He is a Professor with Northwestern Polytechnical University and Harbin Engineering University. His current research interests include pervasive computing, mobile crowdsensing, internet of things, and intelligent information technology.